
Développement d'applications distribuées

CER | JWT Docker

Lundi 08 juin 2026

TABLE DES MATIÈRES

AAV	2
Étude de cas : Front-end	3
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	4
Livrables	4
Pistes de solutions	5
Plan d'actions	5
Définition : Mots clés	6
JSON	6
Composant	6
React	6
Front-end	7
Wireframe	7
Redux	7
Tailwind CSS	8
useEffect	8
Axios	9
DOM	9
Analyse et résolution : Front-end	10
Conclusion	11
Capitalisation	12
Ressources	13

- 5 Appliquer les règles de création des composants
- 6 Construire et manipuler des données en JSON
- 2 Reconnaître les cycles de vie d'un composant
- 3 Réaliser la modélisation d'une application Frontend
- 5 Utiliser un framework Frontend
- 5 Utiliser l'une des librairies de style (Bootstrap, Tailwind, etc.)

ÉTUDE DE CAS : FRONT-END

MOTS CLÉS

- Json
- Composant
- React
- Front-end
- Wireframe
- Redux
- Tailwind CSS
- useEffect
- Axios
- DOM

CONTEXTE

Yanis développe le front-end d'une application avec des composants modulaires via React et Tailwind CSS. Il rencontre des erreurs d'initialisation car les données JSON requises par son composant ne sont pas complètement chargées au moment du rendu de la page.

PROBLÉMATIQUE

Comment gérer efficacement le cycle de vie des composants et le chargement asynchrone des données dans une application React pour éviter les erreurs d'affichage ?

CONTRAINTES

- React
- Tailwind CSS
- JSON

GÉNÉRALISATION

- Front-end
- Composant

LIVRABLES

- Diagramme séquence
- Code source de l'application Front-end

PISTES DE SOLUTIONS

- Se renseigner sur le cycle de vie des composants React et les hooks useEffect
- Se renseigner sur Redux
- Concevoir des composants indépendants et réutilisables

PLAN D' ACTIONS

- DMC
- ER
- Étudier le fonctionnement de React (composants, hooks, états)
- Voir l'architecture de l'application via des wireframes.
- Initialiser un projet React/Next.js.
- Créer les premiers composants
- Installer et configurer Axios
- Implémenter les hooks (useState, useEffect) pour gérer correctement les données et le cycle de vie des composants.
- Gérer la navigation et le routage entre les différentes pages de l'appli

DÉFINITION : MOTS CLÉS

JSON

JSON (*JavaScript Object Notation*) est un **format d'échange de données** léger, textuel et lisible par l'humain, devenu le standard de la communication entre un front-end et une API. Il repose sur deux structures : des **objets** (ensembles de paires "clé": valeur entre accolades) et des **tableaux** (listes ordonnées entre crochets). Exemple de réponse d'API :

```
{ "id": 42, "auteur": "Alice", "tags": ["BD", "manga"], "publie": true }
```

En JavaScript, on **manipule** le JSON avec deux fonctions clés : `JSON.parse()` convertit une chaîne JSON reçue en objet JavaScript exploitable, et `JSON.stringify()` fait l'inverse pour l'envoyer au serveur. C'est l'objet de l'AAV « Construire et manipuler des données en JSON » : les données reçues via Axios sont du JSON, transformées en objets puis affichées dans les composants.

Composant

Un **composant** est la **bricke de base, autonome et réutilisable**, d'une interface React. Concrètement, c'est une fonction JavaScript qui reçoit des données en entrée (les **props**) et retourne un morceau d'interface décrit en **JSX** (syntaxe mêlant HTML et JS). Une page se construit en **assemblant** des composants imbriqués, comme des LEGO.

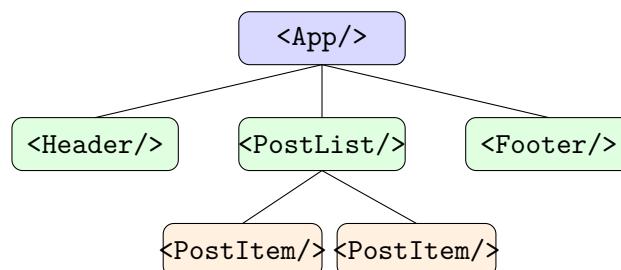


Figure : arbre de composants — une UI React est une hiérarchie de composants réutilisables.

Règles de création (AAV « Appliquer les règles de création des composants ») : nom en **PascalCase** (`PostItem`), **responsabilité unique**, fonction **pure** (mêmes props → même rendu), **props en lecture seule** (jamais modifiées), retour d'un **seul élément racine**, et conception **réutilisable et indépendante**.

React

React est une **bibliothèque JavaScript** front-end (créée par Facebook/Meta) servant à construire des interfaces utilisateur à base de **composants**. C'est le **framework Frontend** retenu pour le projet (AAV « Utiliser un framework Frontend »). Ses principes fondateurs :

- **Déclaratif** : on décrit l'interface en fonction de l'état ; React se charge de mettre à jour l'affichage.
- **Composants** : l'UI est découpée en briques réutilisables.
- **Virtual DOM** : React maintient une copie virtuelle du DOM et ne met à jour que ce qui change réellement, pour de bonnes **performances**.
- **Flux de données unidirectionnel** : les données descendent du parent vers l'enfant via les props.
- **Hooks** : fonctions (useState, useEffect...) qui ajoutent état et cycle de vie aux composants.

Front-end

Le **front-end** désigne la **partie cliente** d'une application : tout ce qui s'exécute dans le **navigateur** de l'utilisateur et avec quoi il interagit directement (l'interface, l'affichage, l'expérience utilisateur). Il est construit avec **HTML** (structure), **CSS** (style) et **JavaScript** (interactivité), ici via React et Tailwind. Il s'oppose au **back-end** (côté serveur : logique métier, base de données, API) avec lequel il dialogue en échangeant du **JSON** via des requêtes HTTP.

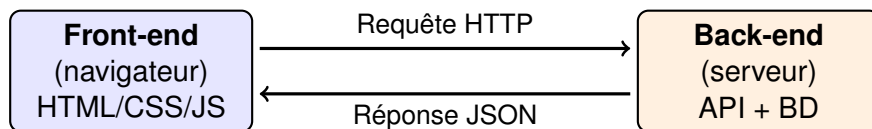


Figure : le front-end (client) dialogue avec le back-end (serveur) via HTTP/JSON.

Wireframe

Un **wireframe** (« maquette filaire ») est un **schéma simplifié, en basse fidélité**, de la structure d'une page ou d'un écran. Sans couleurs ni graphismes définitifs, il sert à positionner les **zones fonctionnelles** (en-tête, navigation, contenu, pied de page) et à valider l'**agencement** avant de coder. C'est un outil central de la **modélisation d'une application Frontend** (AAV) et un préalable au découpage en composants.

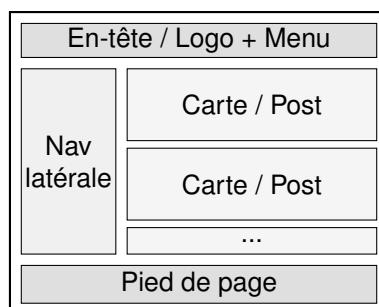


Figure : wireframe d'une page — la structure est définie avant le style et le code.

Redux

Redux est une bibliothèque de **gestion d'état global** pour les applications JavaScript (souvent associée à React). Quand un état doit être **partagé entre de nombreux**

composants éloignés (utilisateur connecté, panier, liste de posts), le faire « remonter » via les props devient ingérable. Redux centralise cet état dans un **store unique** et impose un **flux de données unidirectionnel et prévisible** :

- **Store** : source de vérité unique contenant tout l'état.
- **Action** : objet décrivant « ce qui s'est passé » (ex. : { type: 'ADD_POST' }).
- **Reducer** : fonction pure qui calcule le **nouvel état** à partir de l'état courant et de l'action.
- **Dispatch** : déclenche une action depuis l'interface.

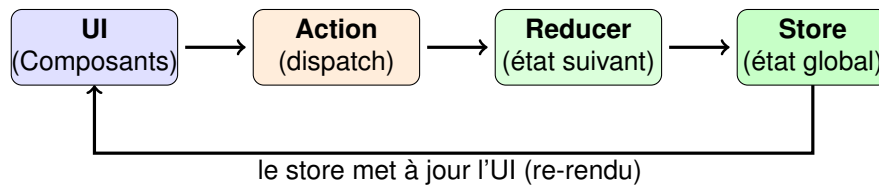


Figure : flux unidirectionnel de Redux — UI → action → reducer → store → UI.

Tailwind CSS

Tailwind CSS est un framework CSS dit « **utility-first** » : au lieu de fournir des composants pré-stylés (comme Bootstrap), il offre une multitude de **classes utilitaires** atomiques que l'on compose directement dans le HTML/JSX pour styliser élément par élément. Exemple :

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">Envoyer</button>
```

Ici `bg-blue-600` (fond bleu), `text-white` (texte blanc), `px-4 py-2` (marges internes), `rounded` (coins arrondis). **Avantages** : rapidité, cohérence visuelle, design **responsive** simple (`md:`, `lg:`), pas de feuilles CSS séparées à maintenir. C'est la **librairie de style** retenue (AAV « Utiliser l'une des librairies de style »). **Différence avec Bootstrap** : Bootstrap impose des composants tout faits (look reconnaissable), Tailwind laisse une liberté totale de design.

useEffect

`useEffect` est le **hook** React qui permet d'exécuter des **effets de bord** (actions extérieures au rendu) : appels à une API, abonnements, minuteurs, manipulation directe du DOM. Il est **central pour gérer le cycle de vie** d'un composant (AAV « Reconnaître les cycles de vie d'un composant ») et répond directement à la **problématique du projet** (chargement asynchrone des données). Son comportement dépend du **tableau de dépendances** :

- `useEffect(fn, [])` : s'exécute **une seule fois au montage** — idéal pour charger les données initiales.
- `useEffect(fn, [x])` : se ré-exécute à chaque changement de `x` (mise à jour).
- **Fonction de nettoyage** (`return`) : exécutée au démontage, pour annuler abonnements/requêtes et éviter les fuites.



Figure : les trois phases du cycle de vie pilotées par *useEffect*. Dans le projet, on charge les données dans un `useEffect([])` et on affiche un état de chargement tant que le JSON n'est pas arrivé — ce qui élimine les erreurs d'initialisation décrites dans le contexte.

Axios

Axios est une bibliothèque cliente **HTTP basée sur les promesses** (*Promise*) permettant au front-end d'effectuer des **requêtes asynchrones** vers une API REST (GET, POST, PUT, DELETE) et d'en récupérer les données. Atouts par rapport au `fetch` natif : conversion **automatique en JSON**, gestion simplifiée des erreurs, en-têtes et intercepteurs, délais d'attente. Couplé à *useEffect*, il constitue le mécanisme de **chargement asynchrone** des données :

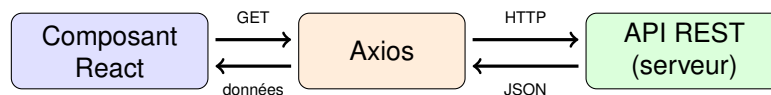


Figure : *Axios* fait le pont asynchrone entre le composant *React* et l'*API REST*.

DOM

Le **DOM** (*Document Object Model*) est la **représentation en mémoire, sous forme d'arbre**, de la page HTML telle qu'interprétée par le navigateur. Chaque balise HTML y devient un **nœud** manipulable par JavaScript (créer, modifier, supprimer des éléments, réagir aux événements). C'est via le DOM que l'interface est réellement affichée et mise à jour.

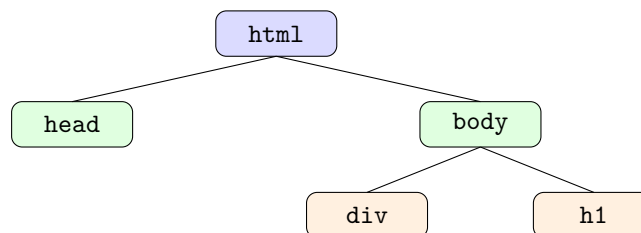


Figure : le DOM, arbre des nœuds de la page. React n'agit pas directement sur ce DOM réel (coûteux) : il passe par un **Virtual DOM**, en calcule les différences, et ne met à jour que les nœuds modifiés — d'où ses bonnes **performances**.

ANALYSE ET RÉOLUTION : FRONT-END

CONCLUSION

En réponse à la problématique, gérer efficacement le cycle de vie des composants et le chargement asynchrone des données dans une application React repose sur une combinaison cohérente de bonnes pratiques. L'interface est découpée en **composants modulaires, indépendants et réutilisables** (règles de création : PascalCase, responsabilité unique, props en lecture seule), dont la conception est préparée en amont par des **wireframes** qui modélisent la structure de l'application avant le codage.

Le cœur de la solution réside dans la maîtrise du **cycle de vie** via le hook `useEffect` : les données sont chargées de façon **asynchrone** à l'aide d'**Axios** dans un effet déclenché au montage du composant (`[]`), et l'on affiche un **état de chargement** tant que le **JSON** n'est pas arrivé. Cette approche supprime les **erreurs d'initialisation** décrites dans le contexte (rendu déclenché avant la disponibilité des données). Une **fonction de nettoyage** au démontage évite par ailleurs les fuites mémoire et les mises à jour sur composant démonté.

Pour l'état partagé entre de nombreux composants, **Redux** centralise les données dans un store unique au flux unidirectionnel et prévisible, tandis que **Tailwind CSS** assure un style cohérent et responsive. Au final, la stack **React + Axios + Redux + Tailwind** satisfait l'ensemble des AAV visés (création de composants, manipulation de JSON, cycle de vie, modélisation, framework et librairie de style) et constitue un **modèle reproductible** pour le développement front-end des futures applications de l'entreprise.

CAPITALISATION

AAV	Commentaire	Statut
Appliquer les règles de création des composants	Règles détaillées (PascalCase, responsabilité unique, fonction pure, props en lecture seule, racine unique, réutilisabilité) et illustrées par un arbre de composants.	Acquis
Construire et manipuler des données en JSON	Structure du JSON (objets/tableaux) expliquée ; manipulation via <code>JSON.parse/JSON.stringify</code> et exploitation des réponses d'API.	Acquis
Reconnaître les cycles de vie d'un composant	Phases montage / mise à jour / démontage schématisées et pilotées par <code>useEffect</code> (tableau de dépendances, nettoyage).	Acquis
Réaliser la modélisation d'une application Frontend	Modélisation par wireframes (structure des écrans) et découpage en arbre de composants présentés et schématisés.	Acquis
Utiliser un framework Frontend	React expliqué (composants, déclaratif, Virtual DOM, flux unidirectionnel, hooks) et retenu comme framework du projet.	Acquis
Utiliser l'une des bibliothèques de style (Bootstrap, Tailwind, etc.)	Tailwind CSS (utility-first) présenté avec exemple de classes, design responsive et comparaison avec Bootstrap.	Acquis

RESSOURCES

- **Documentation React** : <https://react.dev/>
- **Hook useEffect (cycle de vie)** : <https://react.dev/reference/react/useEffect>
- **Documentation Tailwind CSS** : <https://tailwindcss.com/docs>
- **Documentation Redux / Redux Toolkit** : <https://redux.js.org/>
- **Documentation Axios** : <https://axios-http.com/fr/docs/intro>
- **JSON (MDN)** : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/JSON
- **DOM (MDN)** : https://developer.mozilla.org/fr/docs/Web/API/Document_Object_Model
- **Next.js (framework React)** : <https://nextjs.org/docs>